Spatial batch normalization

In fully connected networks, we performed batch normalization on the activations. To do something equivalent on CNNs, we modify batch normalization slightly.

Normally batch-normalization accepts inputs of shape `(N, D)` and produces outputs of shape `(N, D)`, where we normalize across the minibatch dimension `N`. For data coming from convolutional layers, batch normalization accepts inputs of shape `(N, C, H, W)` and produces outputs of shape `(N, C, H, W)` where the `N` dimension gives the minibatch size and the `(H, W)` dimensions give the spatial size of the feature map.

How do we calculate the spatial averages? First, notice that for the `C` feature maps we have (i.e., the layer has `C` filters) that each of these ought to have its own batch norm statistics, since each feature map may be picking out very different features in the images. However, within a feature map, we may assume that across all inputs and across all locations in the feature map, there ought to be relatively similar first and second order statistics. Hence, one way to think of spatial batch-normalization is to reshape the `(N, C, H, W)` array as an `(N*H*W, C)` array and perform batch normalization on this array.

Since spatial batch norm and batch normalization are similar, it'd be good to at this point also copy and paste our prior implemented layers from HW #4. Please copy and paste your prior implemented code from HW #4 to start this assignment. If you did not correctly implement the layers in HW #4, you may collaborate with a classmate to use their implementations from HW #4. You may also visit TA or Prof OH to correct your implementation.

You'll want to copy and paste from HW #4: - layers.py for your FC network layers, as well as batchnorm and dropout. - layer_utils.py for your combined FC network layers. - optim.py for your optimizers.

Be sure to place these in the `nndl/` directory so they're imported correctly. Note, as announced in class, we will not be releasing our solutions.

If you use your prior implementations of the batchnorm, then your spatial batchnorm implementation may be very short. Our implementations of the forward and backward pass are each 6 lines of code.

CS231n has built a solid API for building these modular frameworks and training them, and we will use their very well implemented framework as opposed to "reinventing the wheel." This includes using their Solver, various utility functions, their layer structure, and their implementation of fast CNN layers. This also includes nndl.fc_net, nndl.layers, and nndl.layer_utils. As in prior assignments, we thank Serena Yeung & Justin Johnson for permission to use code written for the CS 231n class (cs231n.stanford.edu).

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from nndl.conv_layers import spatial_batchnorm_forward, spatial_batchnorm
from cs231n.gradient_check import eval_numerical_gradient_array

%matplotlib inline
plt.rcParams["figure.figsize"] = (10.0, 8.0)  # set default size of plots
plt.rcParams["image.interpolation"] = "nearest"
plt.rcParams["image.cmap"] = "gray"

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in
%load_ext autoreload
%autoreload 2


def rel_error(x, y):
    """returns relative error"""
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y)
```

Spatial batch normalization forward pass

Implement the forward pass, `spatial_batchnorm_forward` in `nndl/conv_layers.py`. Test your implementation by running the cell below.

In [2]:
```
# Check the training-time forward pass by checking means and variances
# of features both before and after spatial batch normalization

N, C, H, W = 2, 3, 4, 5
x = 4 * np.random.randn(N, C, H, W) + 10

print("Before spatial batch normalization:")
print(f"    Shape: {x.shape}")
print(f"    Means: {x.mean(axis=(0, 2, 3))}")
print(f"    Stds: {x.std(axis=(0, 2, 3))}")

# Means should be close to zero and stds close to one
gamma, beta = np.ones(C), np.zeros(C)
bn_param = {"mode": "train"}
out, _ = spatial_batchnorm_forward(x, gamma, beta, bn_param)
print("After spatial batch normalization:")
print(f"    Shape: {out.shape}")
print(f"    Means: {out.mean(axis=(0, 2, 3))}")
print(f"    Stds: {out.std(axis=(0, 2, 3))}")

# Means should be close to beta and stds close to gamma
gamma, beta = np.asarray([3, 4, 5]), np.asarray([6, 7, 8])
out, _ = spatial_batchnorm_forward(x, gamma, beta, bn_param)
print("After spatial batch normalization (nontrivial gamma, beta):")
print(f"    Shape: {out.shape}")
print(f"    Means: {out.mean(axis=(0, 2, 3))}")
print(f"    Stds: {out.std(axis=(0, 2, 3))}")
```

```
Before spatial batch normalization:
    Shape: (2, 3, 4, 5)
    Means: [11.39389058 10.10571223 10.09492495]
    Stds: [3.96734762 3.74180461 4.18861633]
After spatial batch normalization:
    Shape: (2, 3, 4, 5)
    Means: [5.68989300e-16 2.83106871e-16 2.91433544e-16]
    Stds: [0.99999968 0.99999964 0.99999972]
After spatial batch normalization (nontrivial gamma, beta):
    Shape: (2, 3, 4, 5)
    Means: [6. 7. 8.]
    Stds: [2.99999905 3.99999857 4.99999858]
```

Spatial batch normalization backward pass

Implement the backward pass, `spatial_batchnorm_backward` in `nndl/conv_layers.py`. Test your implementation by running the cell below.

In [3]:
```
N, C, H, W = 2, 3, 4, 5
x = 5 * np.random.randn(N, C, H, W) + 12
gamma = np.random.randn(C)
beta = np.random.randn(C)
dout = np.random.randn(N, C, H, W)

bn_param = {"mode": "train"}


def fx(x):
    return spatial_batchnorm_forward(x, gamma, beta, bn_param)[0]


def fg(a):
    return spatial_batchnorm_forward(x, gamma, beta, bn_param)[0]


def fb(b):
    return spatial_batchnorm_forward(x, gamma, beta, bn_param)[0]


dx_num = eval_numerical_gradient_array(fx, x, dout)
da_num = eval_numerical_gradient_array(fg, gamma, dout)
db_num = eval_numerical_gradient_array(fb, beta, dout)

_, cache = spatial_batchnorm_forward(x, gamma, beta, bn_param)
dx, dgamma, dbeta = spatial_batchnorm_backward(dout, cache)
print(f"dx error: {rel_error(dx_num, dx)}")
print(f"dgamma error: {rel_error(da_num, dgamma)}")
print(f"dbeta error: {rel_error(db_num, dbeta)}")
```

```
dx error: 1.5529538968779867e-08
dgamma error: 1.5333173303285033e-10
dbeta error: 9.381226834152662e-12
```

Question

In our architecture, we apply batch normalization *before* ReLU. What would change if we applied it *after* ReLU instead? Which ordering do you think is better, and why?

Answer

What would change if we applied Batch Normalization after ReLU: If we apply Batch Normalization after ReLU, the input to the BN layer will be strictly non-negative (since ReLU outputs $\max(0, x)$). The distribution of these activations is non-Gaussian and heavily skewed, with a massive spike at zero. When BN attempts to zero-center this data, it will calculate a positive mean and subtract it from all values. As a result, the previously zeroed-out (deactivated) neurons will be shifted to negative values. This alters the fundamental sparse, non-negative property that ReLU was meant to introduce.

Which ordering is better and why: Applying Batch Normalization **before** ReLU is generally considered the better approach (and is the standard practice recommended in the original Batch Normalization paper by Ioffe and Szegedy).

Here is why:

1. **Appropriate Distribution for BN:** Batch Normalization assumes that the data it is normalizing is roughly symmetric and Gaussian. The output of an affine or convolutional layer (a weighted sum) fits this assumption much better than the truncated, skewed output of a ReLU layer.
2. **Controlling Activation:** By applying BN before ReLU, the network ensures that the inputs to the activation function are stable and well-scaled. The learnable parameters (γ and β) in the BN layer can then optimally shift and scale this symmetric distribution to control exactly what proportion of the values pass through the ReLU (i.e., are greater than zero).
3. **Preserving Sparsity:** ReLU introduces representational sparsity by completely turning off certain neurons (outputting exactly 0). If BN is applied after ReLU, it destroys this sparsity by shifting those exact zeros into negative numbers, forcing the next layer to deal with a completely different input landscape.